

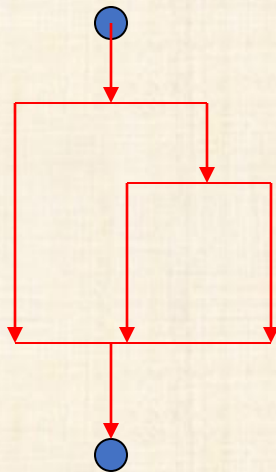
第四章 汇编语言程序设计

顺序程序、分支程序、循环程序、
查表程序、子程序、算术运算

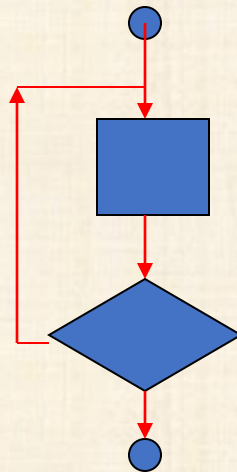




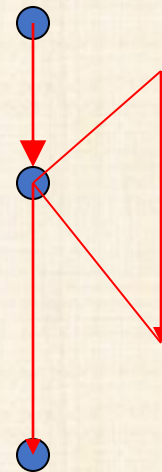
顺序结构



分支结构



循环结构



子程序

4.1 顺序程序设计

- 特点：程序按先后次序执行，程序简单。

顺序结构程序是最简单的程序结构。程序既无分支、循环，也不调用子程序，程序执行时一条接一条地按顺序执行指令。

- 例：3字节无符号数相加，其中被加数在内部RAM的50H、51H和52H单元中；加数在内部RAM的53H、54H和55H单元中；要求把相加之和存放在50H、51H和52H单元中，进位存放在位寻址区的20H位中。

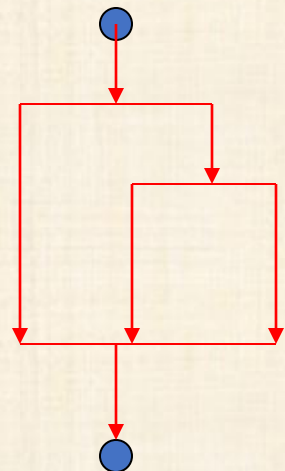
- ORG 0000H
- MOV R0,#52H
- MOV R1,#55H
- MOV A,@R0
- ADD A,@R1
- MOV @R0,A
- DEC R0
- DEC R1
- MOV A,@R0
- ADDC A,@R1
- MOV @R0,A
- DEC R0
- DEC R1
- MOV A,@R0
- ADDC A,@R1
- MOV @R0,A
- CLR A
- ADDC A,#00H
- MOV R0,#20H
- MOV @R0,A
- SJMP \$
- END

4.2 分支程序设计

- 方法： 1)采用条件转移指令

2)二分支

JZ,JNZ; JC,JNC; JB,JNB; CJNE;



- 单分支程序设计

例：假定在外部RAM中有ST1、ST2和ST3共3个连续单元，其中ST1和ST2单元中存放着两个无符号二进制数，要求找出其中的大数并存入ST3单元中。

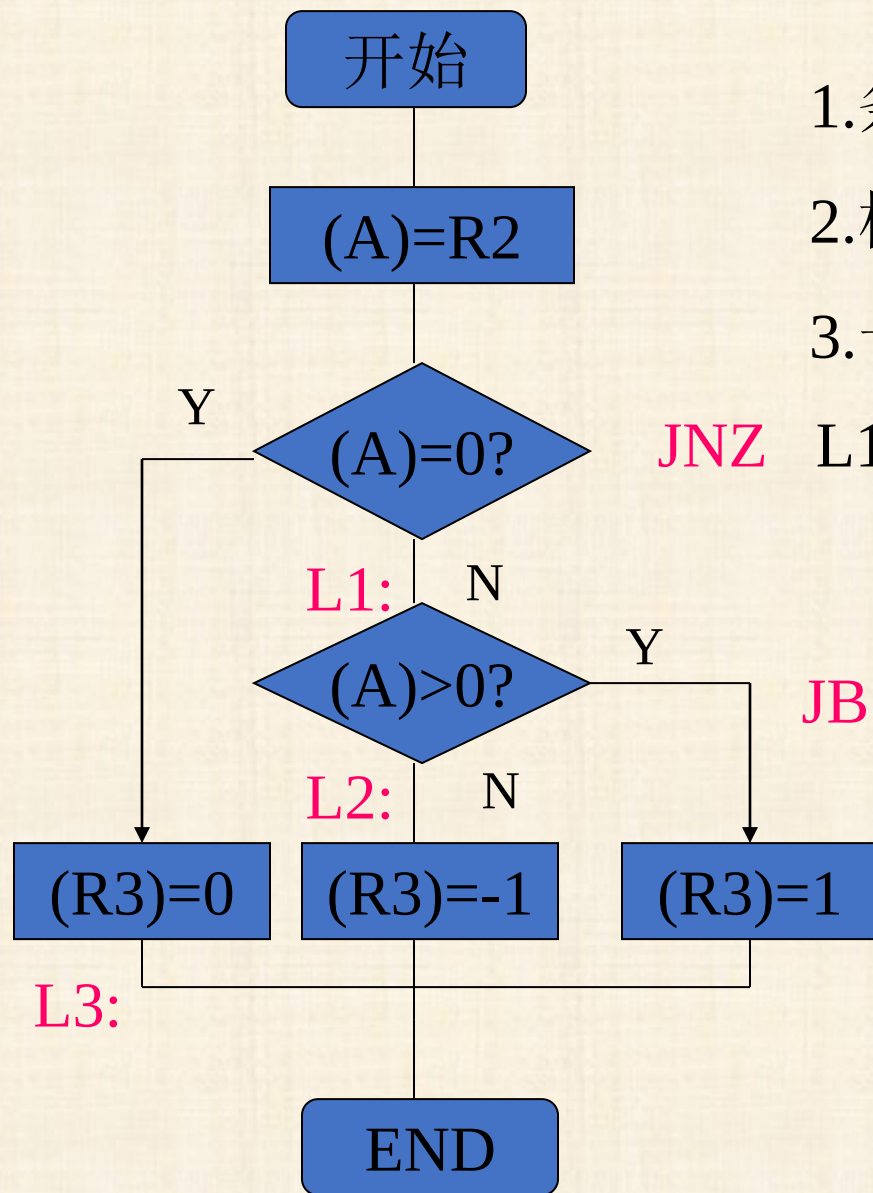
```
START:  CLR C                ;进位位清0
        MOV DPTR, #ST1      ;设置数据指针
        MOVX A, @DPTR       ;取第1个数
        MOV R2, A           ;第1个数存于R2
        INC DPTR            ;数据指针加1
        MOVX A, @DPTR       ;取第2个数
        SUBB A, R2           ;两数比较
        JNC BIG1            ;若第2个数大, 则转向BIG1
        XCH A, R2           ;若第1个数大, 则整字节交换
BIG0:   INC DPTR
        MOVX @DPTR, A       ;存大数
        RET
BIG1:   MOVX A, @DPTR
        SJMP BIG0
```

- 多分支程序设计

例：设变量X存放于R2， Y存放在R3，
试按下式给Y赋值。

$$Y = \begin{cases} 1 & \text{---} X > 0 \\ 0 & \text{---} X = 0 \\ -1 & \text{---} X < 0 \end{cases}$$

1. 条件语句
2. 标号
3. 一路一路写



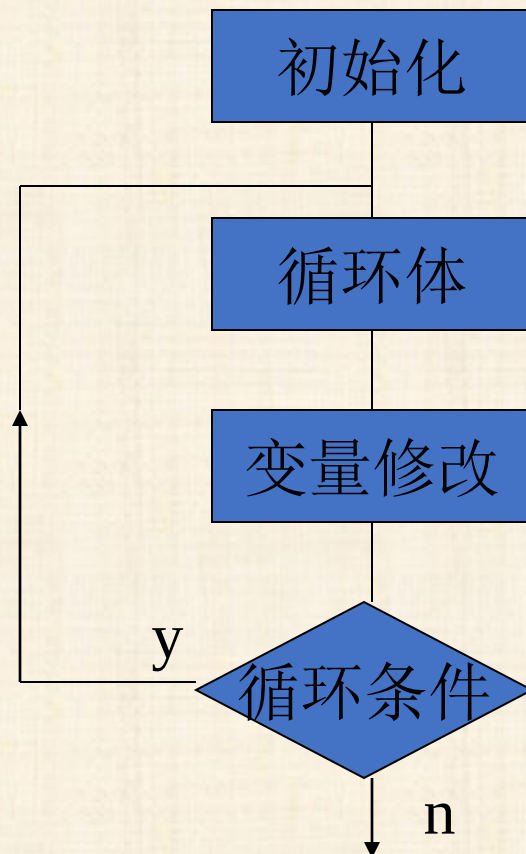
流程图

- MOV A,R2
- JNZ L1
- MOV R3,#0
- SJMP L3
- L1: JB ACC.7, L2
- MOV R3, #1
- SJMP L3
- L2: MOV R3, #0FFH
- L3: SJMP L3
- END

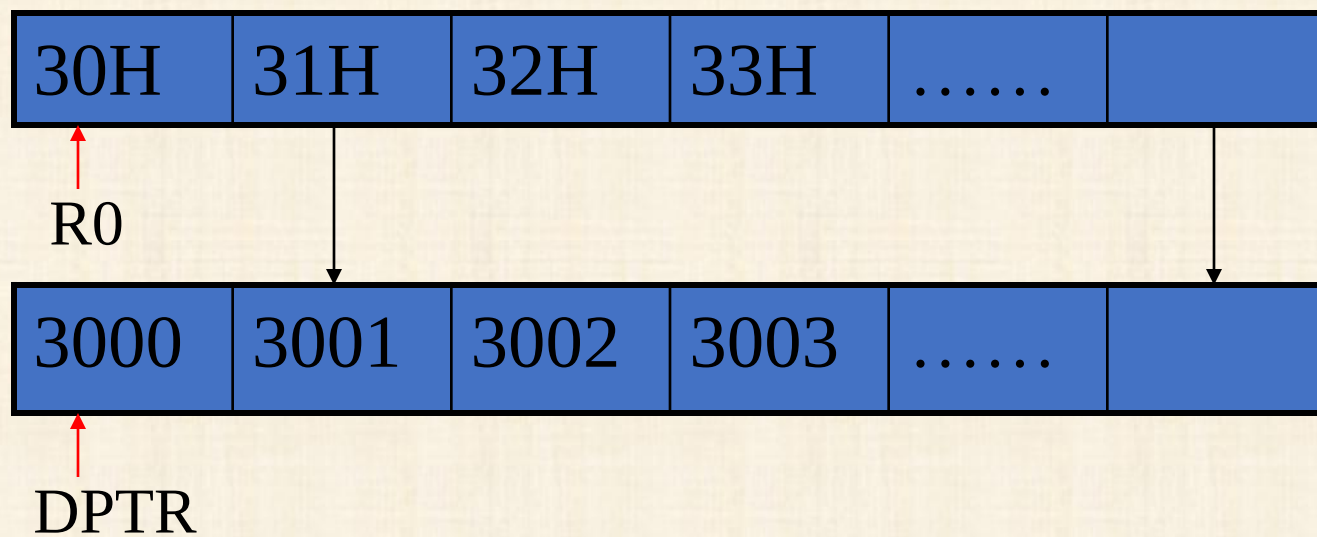
4.3 循环程序设计

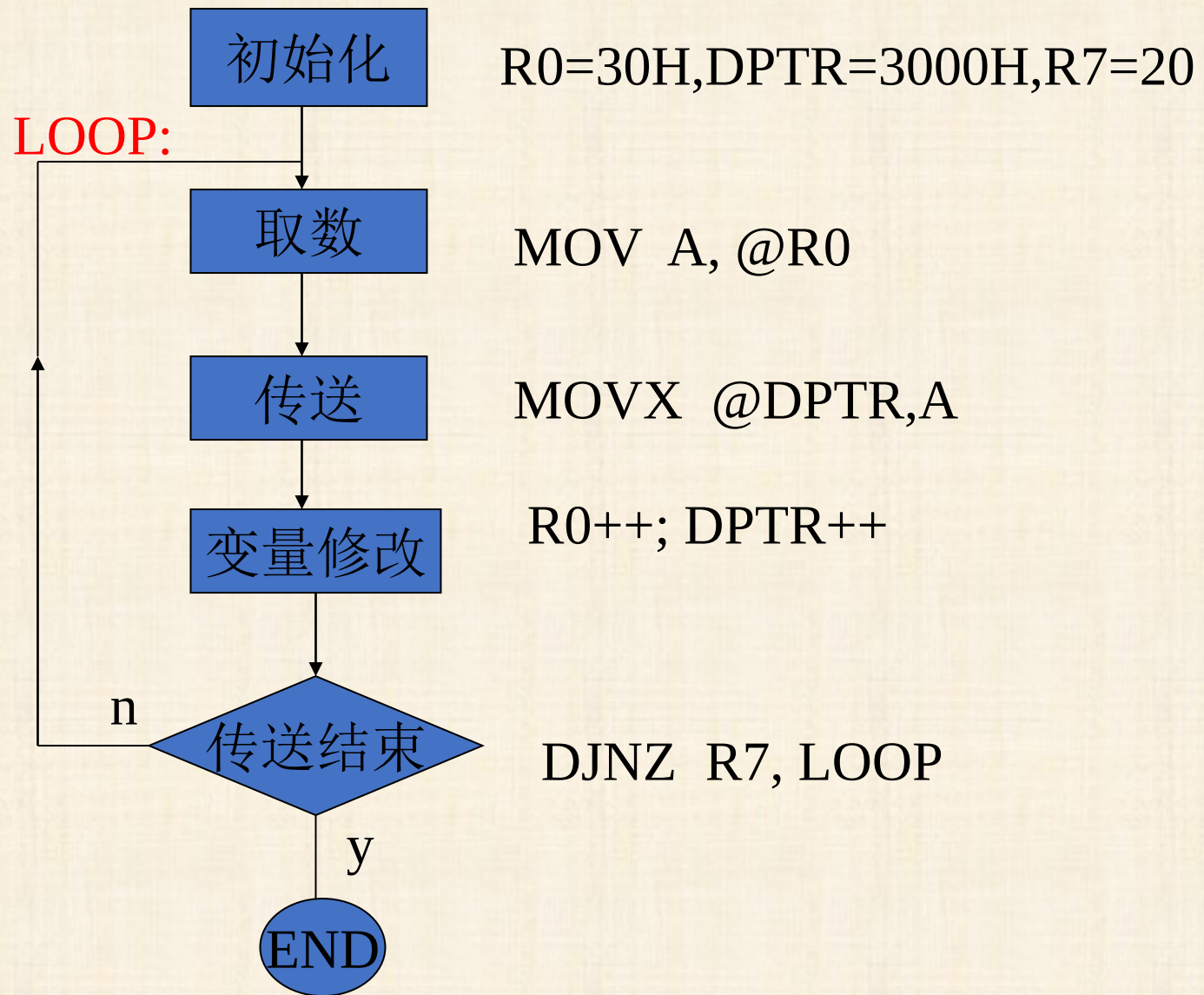
重复执行部分指令。

- 1) 结构
- 2) 数据块处理时,
采用间接寻址
- 3) 计数循环使用
DJNZ 指令



- **例1:** 将内部RAM30H为首地址的20个数据传送到外部RAM3000H开始的单元。





- MOV R0, #30H
- MOV DPTR, #3000H
- MOV R7, #20
- LOOP: MOV A, @R0
- MOVBX @DPTR,A
- INC R0
- INC DPTR
- DJNZ R7, LOOP
- HERE: SJMP HERE

4.4 子程序设计

1. 子程序结构:

子程序名称: 标号

子程序返回: RET

2. 入口参数; 出口参数

3. 子程序调用:

LCALL SUB

ACALL SUB

4. 寄存器保护: 利用堆栈

例1: 编程实现 $c=a^2+b^2$

- XA DATA 40H
- XB DATA 41H
- XC DATA 42H
- MOV A, XA
- ACALL SQR
- MOV XC, A
- MOV A, XB
- ACALL SQR
- ADD A, XC
- MOV XC, A
- SJMP \$
- **SQR: MOV B, A**
- **MUL AB**
- **RET**

例2: 多重循环（定时程序）

5ms延时程序，设fosc=6MHz

- DELAY: MOV R6, #5
- DEL1: MOV R7, #250
- DEL2: DJNZ R7, DEL2
- DJNZ R6, DEL1
- RET

4.5 查表程序设计

1.表格的定义:

使用伪指令: DB,DW

例: TAB: DB 3FH, 06H, 5BH, 4FH, 66H
DB 6DH,7DH, 07H, 7FH, 6FH
DB 77H, 7CH

2.表格的访问:

MOVC A, @A+DPTR

MOVC A, @A+PC

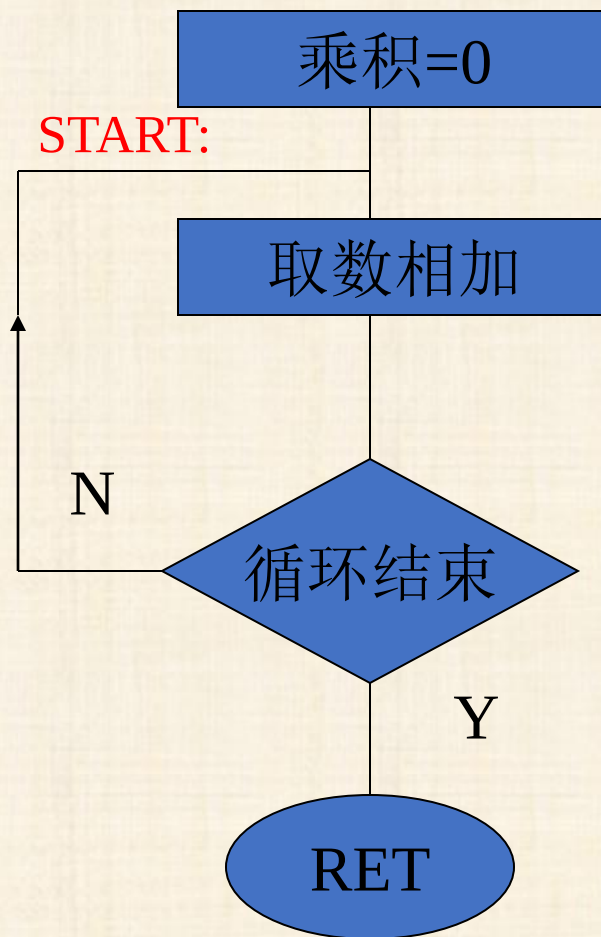
例：已知R0低4位有一个十六进制数（0-F），编程转换成相应的ASCII码并送入R0.

- MOV A, R0
- ANL A, #0FH
- ADD A, #03H
- MOVC A, @A+PC
- MOV R0,A
- SJMP \$
- ASCTAB: DB '0','1','2','3','4','5','6','7'
- DB '8','9','A','B','C','D','E','F'

4.6 运算程序

- 例1:双字节乘法
- $//R7R6 * R5R4 = @R0$
- 入口参数: R7R6, R5R4, R0-积的低字节地址
- 出口参数: R0-积的高字节地址

- 算法1： R7R6连加运算， 次数R5,R4



$$\begin{array}{r}
 (33H)(32H)(31H)(30H) \\
 + 00H \quad 00H \quad R7 \quad R6 \\
 \hline
 (33H)(32H)(31H)(30H)
 \end{array}$$

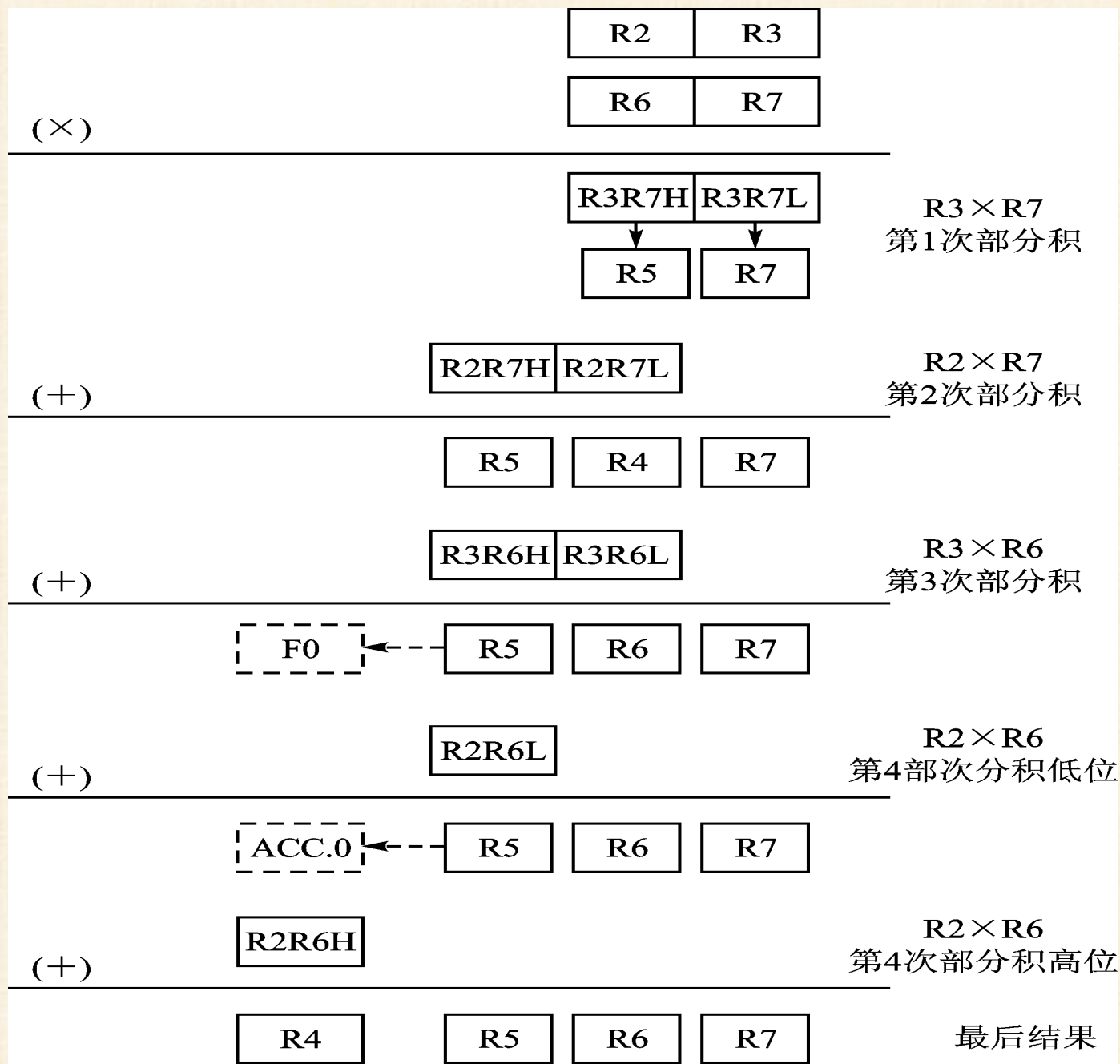
R5R4-1=0?

- ORG 0000H
- START0: MOV 30H, #00H
- MOV 31H, #00H
- MOV 32H, #00H
- MOV 33H, #00H
- START: MOV R0,#30H
- MOV A,@R0
- ADD A,R6
- MOV @R0,A
- INC R0
- MOV A,@R0
- ADDC A,R7
- MOV @R0,A
- INC R0
- MOV A,@R0

- ADDC A,#0
- MOV @R0,A
- INC R0
- MOV A,@R0
- ADDC A,#0
- MOV @R0,A
- CLR C
- CJNE R5,#0,BB
- MOV A,R4
- SUBB A,#1
- MOV R4,A
- JNZ START
- SJMP CC

- BB: MOV A,R4
- SUBB A,#1
- MOV R4,A
- MOV A,R5
- SUBB A,#0
- MOV R5,A
- JNC START
- CC: RET

• 算法2： 双字节数乘法(P84) $R2R3 \times R6R7 = R4R5R6R7$

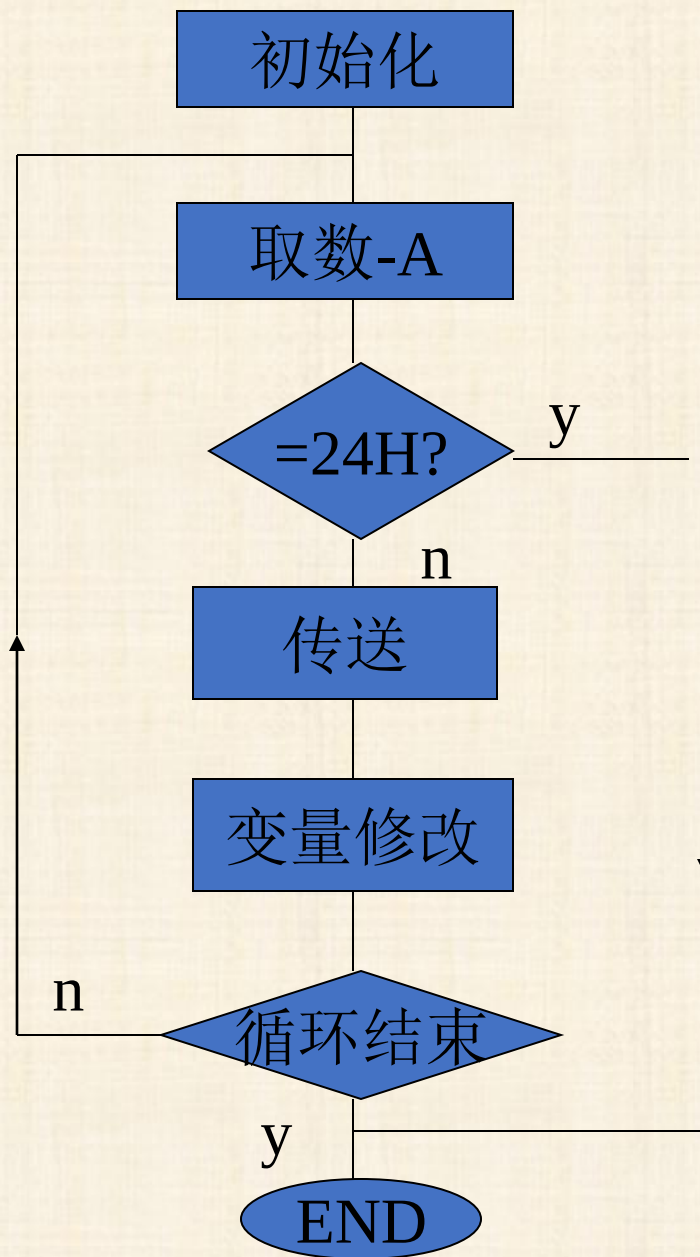


DBMUL: MOV A, R3
MOV B, R7
MUL AB; $R3 \times R7$ (得第1次部分积)
XCH A, R7; 原R7内容送
A, $R7 \leftarrow R3R7L$ (在R7中得到乘积
的第4字节)
MOV R5, B; $R5 \leftarrow R3R7H$
MOV B, R2
MUL AB; $R2 \times R7$ (得第2次部分积)
ADD A, R5; $R2R7L + R3R7H$
MOV R4, A; $R4 \leftarrow$ 和
CLR A
ADDC A, B; $R2R7H + (R2R7L + R5$
时产生的进位)
MOV R5, A; $R5 \leftarrow$ 和
MOV A, R6
MOV B, R3
MUL AB; $R3 \times R6$ (得第3次部分积)
ADD A, R4; $R3R6L + R4$

XCH A, R6
; $A \leftarrow R6, R6 \leftarrow R3R6L + R4$ (在R6中得到乘
积的第3字节)
XCH A, B; $A \leftarrow R3R6H, B \leftarrow R6$
ADDC A, R5
; $R3R6H + R5 + (R3R6L + R4$ 时产生的进位)
MOV R5, A; $R5 \leftarrow$ 和
MOV F0, C; $F0 \leftarrow$ 进位
MOV A, R2
MUL AB; $R2 \times R6$ (得第4次部分积)
ADD A, R5; $R2R6L + (R3R6H + R5$ 时产生
的进位)
MOV R5, A; 在R5中得到乘积的第2字节
CLR A
MOV ACC.0, C; 累加器最高位 $\leftarrow$ 进位
MOV C, F0
ADDC A, B; $R2R6H + F0 + ACC.0$
MOV R4, A; 在R4中得到乘积的第1字节
RET

4.7 综合程序设计

- 例1: 把内部RAM中起始地址为inbuf的数据串传送到外部RAM以outbuf为首地址的区域, 发现“\$”字符的ASCII码停止传送, 数据串最大长度为32字节。
 - 1) 循环程序包含分支程序。
 - 2) 计数循环, 采用间接寻址。



- inbuf EQU 30H
- outbuf EQU 1000H
- MOV R0, #inbuf
- MOV DPTR, #outbuf
- MOV R1, #20H
- LOOP: MOV A, @R0
- CLR C
- SUBB A, #24H
- JZ LOOP1
- MOV A, @R0
- MOVX @DPTR, A
- INC DPTR
- INC R0
- DJNZ R1, LOOP
- LOOP1: RET

- 例2: 在P1.0引脚输出1HZ方波信号。

ORG 0000H

LOOP: MOV R1,#100

WAIT: LCALL DELAY

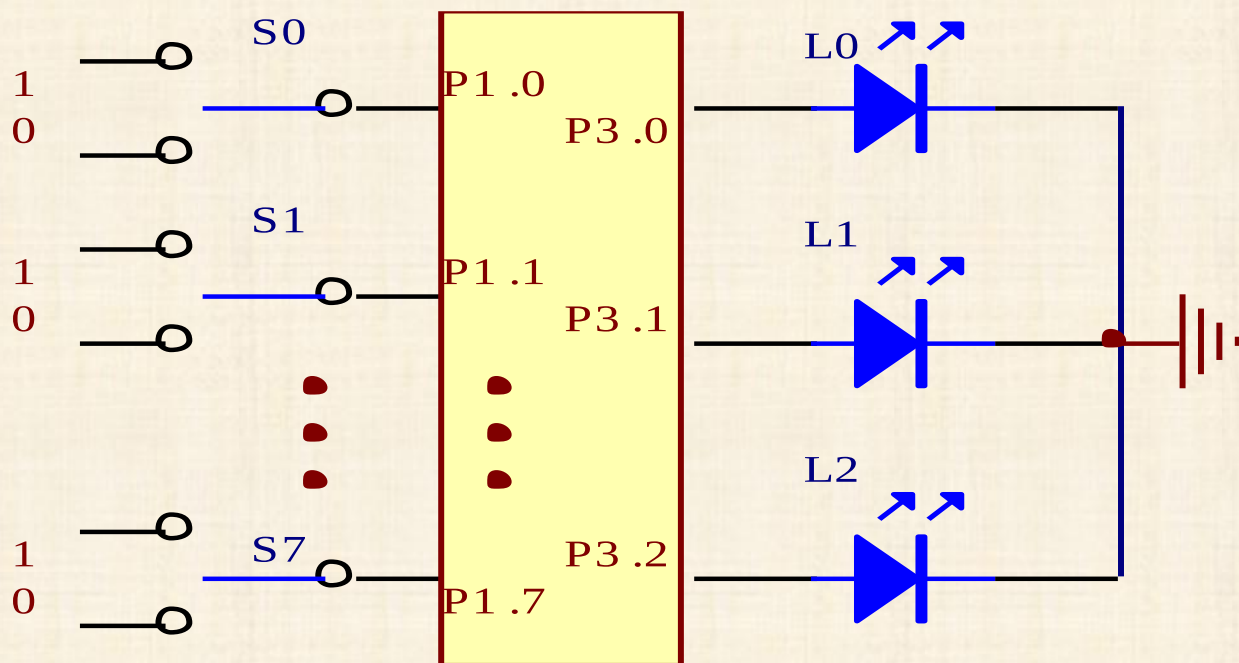
DJNZ R1, WAIT

CPL P1.0

SJMP LOOP

• 例3: 编程实现下面的功能:

$$\left\{ \begin{array}{ll} P3.0 = 1 & P1 < 100 \\ P3.1 = 1 & P1 = 100 \\ P3.2 = 1 & P1 > 100 \end{array} \right.$$



- LED0 EQU P3.0
- LED1 EQU P3.1
- LED2 EQU P3.2
- LOOP: MOV A, P1
- CLR C
- SUBB A,#100
- JZ EQUAL
- JC SMALL
- CLR LED0
- CLR LED1
- SETB LED2
- SJMP OK

- EQUAL: CLR LED0
- SETB LED1
- CLR LED2
- SJMP OK
- SMALL: SETB LED0
- CLR LED1
- CLR LED2
- OK: SJMP LOOP

4.8 C51程序设计

1) C51程序结构

```
void main()  
{   unsigned char x,y;  
    unsigned int z;  
    z=x+(unsigned int)y;   /*z=x+y*/  
    z=(unsigned int)x*y;   //z=x*y  
    printf("result=%d\n",z);  
}
```

#C51程序是一个函数的集合，有一个main的主函数。函数分为库函数和自定义函数。

2) 数据类型

类型	字节数	范围
bit	1/8	0-1
char	1	-128-+127
unsigned char	1	0-255
int	2	-32768-+32767
unsigned int	2	0-65535
long	4	-2147483648-
unsigned long	4	0-4294967295
float	4	$\pm 1.175494\text{E}-38 \sim$ $\pm 3.402823\text{E}+38$

3) 存储器类型

data 直接寻址, 内部RAM:00-7FH

bdata 直接寻址, 内部RAM:20-2FH

idata 间接寻址, 内部RAM:00H-FFH

xdata 外部RAM:0000H-FFFFH

pdata 外部RAM:00H-FFH

code 程序存储区ROM

缺省为data

例: char data x,y,z;
 int idata ii,jj,kk;
 float xdata string[20];

4) 特殊功能寄存器

sfr: 字节寻址的特殊功能寄存器

sfr16: 两个连续寻址的16位值

sbit: 位寻址，特殊功能寄存器中的位

例: sfr PSW=0XD0;
 sfr P0=0X80;
 sfr16 T2=0XCC;
 sbit CY=PSW^7;
 sbit CY=0XD7;

包含文件: #include <reg51.h>

5) C51运算符

1) 算数运算符

$+$ (加法运算) ; $-$ (减法运算) ; $*$ (乘法运算)
 $/$ (除法运算) ; $\%$ (模运算)

2) 关系运算符

$<$ (小于) ; $>$ (大于) ; $<=$ (小于等于)
 $>=$ (大于等于) ; $==$ (等于) ; $!=$ (不等于)

3) 逻辑运算符

$\&\&$ (逻辑与) ; $\|\|$ (逻辑或) ; $!$ (逻辑非NOT)

4) 位操作运算符

$\&$ (与) , $|$ (或) , $\wedge$ (异或)
 $\sim$ (取反) , $<<$ (位左移) , $>>$ (位右移)

6) C51流程控制语句

1) 选择语句if

if(表达式){语句;} //if(p1!=100){c=20;}

if(表达式){语句1;} **else**{语句2; }

2) 循环语句

while语句:

while(表达式){语句;}

do-while语句:

do{语句;}

while(表达式;)

for语句

for(表达式1;表达式2;表达式3;) {语句;}



- LED0 EQU P3.0
- LED1 EQU P3.1
- LED2 EQU P3.2
- LOOP: MOV A, P1
- CLR C
- SUBB A,#100
- JZ EQUAL
- JC SMALL
- CLR LED0
- CLR LED1
- SETB LED2
- SJMP OK
- EQUAL: CLR LED0
- SETB LED1
- CLR LED2
- SJMP OK
- SMALL: SETB LED0
- CLR LED1
- CLR LED2
- OK: SJMP LOOP

```
#include <reg51.h>
```

```
sbit LED0=P3^0;
```

```
sbit LED1=P3^1;
```

```
sbit LED2=P3^2;
```

```
void main(void)
```

```
{
```

```
while(1){
```

```
    if(P1>100){
```

```
        LED0=0;
```

```
        LED1=0;
```

```
        LED2=1;
```

```
    }
```

```
    else if(P1==100){
```

```
        LED0=0;
```

```
        LED1=1;
```

```
        LED2=0;
```

```
    }
```

```
    else{
```

```
        LED0=1;
```

```
        LED1=0;
```

```
        LED2=0;}
```

```
}
```

• 作业5

- 1. 写出指令： 1) 将R0内容送R7; 2) 将位40H送位30H; 3) 将ROM 1000H 送内部RAM 10H, 4) 将内部RAM 20H送外部RAM 2000H, 5) 将内部RAM 20H取反。

- 2. 写出结果

(1) MOV 20H, #78H

MOV A, #89H

ADD A, 20H

DA A

A=_____

(2) MOV A, #89H

MOV R1, #98H

SETB C

ADDC A, R1

CY=___ AC=___ OV=___ P=___.

(3) MOV A, #30

MOV R7, #10H

L1: ADD A, #10

JC L2

DJNZ R7, L1

L2: NOP

A=_____

(4) ORG 8000H

DB 1, 2, 3

DW 4, 5, -6

LBL: SJMP LBL

(8000H)=

(8004H)=

LBL=

实验1 单片机仿真系统

一. 实验目的

- 1. 了解单片机仿真系统的组成及功能
- 2. 掌握WAVE2000仿真软件的使用

二. 实验设备

- 微机; LAB8000实验箱; E/2000S仿真器; 编程器

三. 实验内容

- 1. 仿真系统的连接;
- 2. 汇编程序的编辑;
- 输入并编辑下面的程序:

- ORG 0000H
- MAIN: MOV R0, #30H
- MOV R7, #10H
- CLR A
- LOOP: MOV @R0, A
 INC R0
 INC A
 DJNZ R7, LOOP
 SJMP \$

3. 查看机器码

4. 程序的调试

- 使用“单步”、“跟踪”、“执行到光标处”、“全速运行”、“断点运行”等调试方法对上面的程序进行调试。

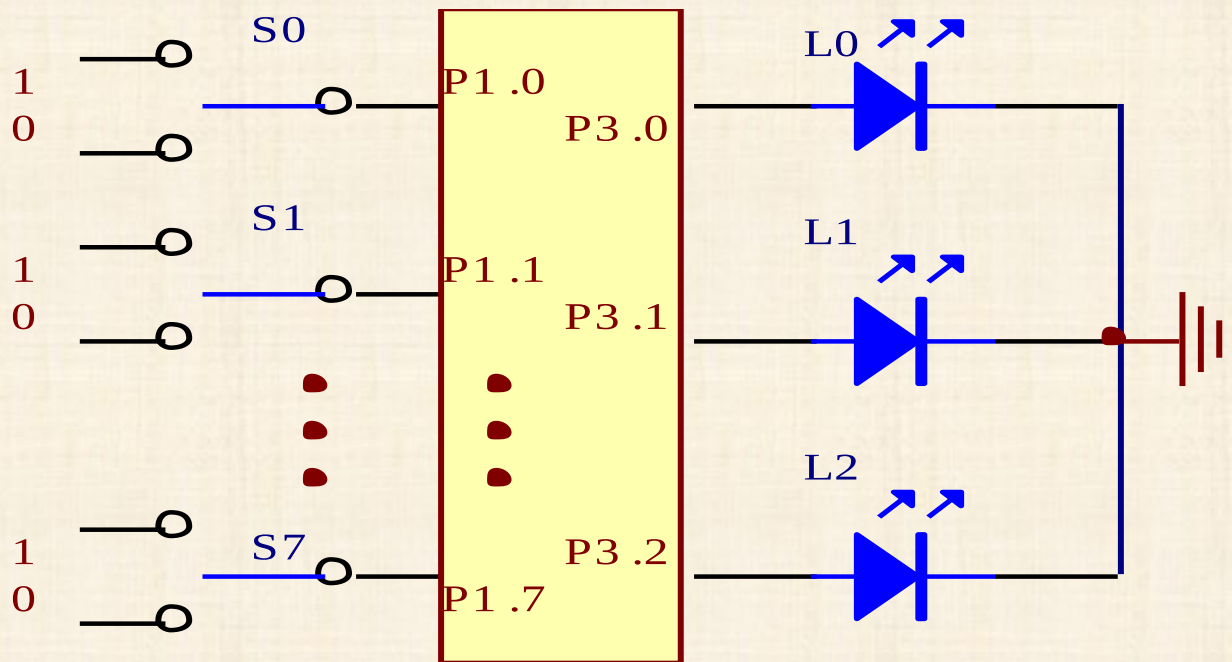
5. 变量及存储器内容的查看及修改

- 在程序运行后，观察寄存器R0, R7, A的变化，观察内部RAM30H-3FH单元的内容变化。

实验2 分支程序设计

- 编程实现下面的功能：

$$\begin{cases} P3.0 = 1 & P1 < 100 \\ P3.1 = 1 & P1 = 100 \\ P3.2 = 1 & P1 > 100 \end{cases}$$



- 南京伟幅
- www.wave-cn.com
- LAB8000实验箱
- WAVE2000 (WAVE6000)